

UrbanFLOW — Comprehensive Technical Research Paper (Final Version + Full Version History)

Author context: This document is the definitive developer-grade technical writeup. It captures the **final shipped production system** (4-model weighted ensemble, HEAD = a18ac71, commit 3f4dca8 = final tree) and the **complete version-by-version lineage** reconstructed from git log, training scripts, benchmark JSONs, and the production predictor.

Every constant, formula, mask, and code snippet below is **verbatim** from the working tree (production_v4.py, train_hydro_gine_v5_0.py, master_pinn_engine.py, active_pinn_pipeline.py, benchmark_*.json). Where something is inferred from commit messages only, it is flagged as such.

Table of Contents

- 1. Executive Summary
- 2. Final Production System (The Whole Logic)
- 3. Dataset & Feature Engineering (Formulas)
- 4. Final Model Architecture — HydroGINE-v5 (Verbatim)
- 5. Final Loss (v5.11, Verbatim)
- 6. The Loss Family — Every Formula Ever Used (Precise)
- 7. Production Physical Continuity Rails (Verbatim)
- 8. The Final Ensemble (Verbatim)
- 9. Full Version History From Git (Including Regressions)
- 10. Benchmark Tables
- 11. Empirical Validation Data Provenance & Source Verification
- 12. Key Lessons / Why Some Versions Became Worse

1. Executive Summary

UrbanFLOW is an autonomous **physics-guided Graph Neural Network (GNN) surrogate** for hyper-local urban flood early warning. It replaces EPA SWMM 5.2 (15–45 min per run) with a sub-100 ms tensor forward pass per city while preserving hydraulic behavior.

Final shipped predictor = EnsembleFloodPredictorV4, a **weighted raw-depth ensemble** of 4 independently-trained HydroGINE checkpoints whose **pre-rail** depths/probabilities are averaged and then passed through the physics rail stack **exactly once**:

Member	Weight	Role in the ensemble
hydro_gine_v5_10_model.pt	0.80	High global precision & speed (the 88.14% leader)
hydro_gine_v5_11_model.pt	0.30	Deep-tail reweighted recovery (lean on deep pools)
hydro_gine_v5_0_model.pt	0.05	HSR / local district recall
hydro_gine_v5_bangalore_opt.pt	0.10	Local valley morphology

Weights normalized by w_sum = 1.25 in __init__.

Final numbers (canonical, data/canonical_metrics.json):

Metric	Value
Global hazard F1 (50 mm/hr)	90.8 % (precision 93.1 %, recall 88.7 %)
Global MAE / RMSE (50 mm/hr)	2.39 cm / 9.13 cm
Catchment NSE	0.9128
Categorical hazard recall @ 480 highest-consequence nodes	99.4 % (74.2 % depth match within ± 15 cm)
Live capture of documented Oct-2024 Bengaluru floods	5 / 13 (38.5 %)
Speedup vs SWMM 5.2	$\sim 1,310\times - 8,690\times$ ($\sim 0.08 - 10$ ms tensor, 85.5 ms avg API)

Legacy live-match metric (doc-specific: risk node $p_depth > 0.08 \text{ m} \wedge |p_depth - y_swmm| < 0.15 \text{ m}$; superseded and **not directly comparable to hazard F1**):

HSR 82.2 %, Bellandur 72.0 %, Berlin / Singapore / Tokyo / Paris / NYC 95.7 / 95.6 / 94.6 / 93.0 / 93.4 %.

Training derivate of final scenario-conditional asymmetric loss + deep-tail reweight lives in `train_hydro_gine_v5_0.py` (OUT = `hydro_gine_v5_11_model.pt`, warm-started from `hydro_gine_v5_10_model.pt`).

2. Final Production System (The Whole Logic)

2.1 Inference path (`production_v4.py`)

```
Request (graph, intensity_mmhr, duration_min)
├─ ProductionFloodPredictorV4.predict() [per ensemble member]
│   ├── feature builder: raw graph → 32-node feature matrix x_full (formulas in §3)
│   ├── normalize x, edge_attr with ckpt stats (x_mean/x_std/e_mean/e_std)
│   ├── model forward → cls_logits, raw_depth (lognormal-domain)
│   ├── p_lin = clamp(expm1(raw_depth * yl_std + yl_mean), 0, 3) [meters]
│   └── p_prob = sigmoid(cls_logits) [flood probability]
├─ Ensemble: p_lin_avg =  $\sum w_i \cdot p\_lin_i$ ; p_prob_avg =  $\sum w_i \cdot p\_prob_i$ 
└─ apply_physics_rails(x_full, edge_index, p_lin_avg, p_prob_avg, elev_range)
    └─ regime classification → confidence gate → mass bound → WSE backwater → clamp
```

Model-format auto-detection (`production_v4.py:282`) — determines whether a checkpoint is v4 or v5 by key presense:

```
ck = torch.load(model_path, map_location=self.device, weights_only=False)
if "convs.0.conv.nn.0.weight" in ck['model']:
    self.model = HydroGINE_v5(...) # GravityGINEConv-backed
else:
    self.model = HydroGINE_v4(...) # plain GINEConv-backed
```

Thresholds: `THR_HAZARD = 0.15 m`, `THR_CRITICAL = 0.30 m` (`production_v4.py:16-17`).

2.2 Live-match metric (the leaderboard quantity)

From training validation `train_hydro_gine_v5_0.py:499-503` and benchmark code:

```
risk node    : p_depth > 0.08 m
live match   : pred classified as risk AND |p_depth - y_swmm| < 0.15 m
match rate   = matches / risk_nodes × 100
over         = pred is risk, swmm y is 0 (false flood)
under        = pred is 0, swmm y > 0 (missed flood)
```

The **production benchmark** applies the rails first, then this definition

(`SELECT_RAIL_MATCH = True` philosophy — raw pre-rail accuracy is only $\sim 47\%$, the rails lift it to $\sim 83\%$ deploy-time).

2.3 Asynchronous serving (app.py, 1431 lines)

/api/predict REST microservice → GeoJSON flood-risk vectors → WebGL/Canvas digital-twin dashboard. Latencies: neural tensor forward 4.0–15.0 ms; full HTTP API 32.0–169.8 ms (avg ≈ 85.0 ms).

3. Dataset & Feature Engineering (Formulas)

3.1 Feature vector — 32 node features

26 physical features (schema in `build_physics_dataset.py`) + **6 derived** features computed at serve time in `production_v4.py`:

Derived features (exact formulas, `production_v4.py:333–340 / 419–422`):

```
total_rain_mm = intensity_mmhr × duration_min / 60

deg_diff      = in_deg - out_deg
dyn_sat       = imp × (1.0 + 0.5 · log1p(intensity × duration / 1000))      # dynamic saturation
true_ponding  = log1p(sink_depth × total_rain_mm / (max(0.2, out_deg) + 0.3)) # ponding index
inflow_load   = expm1(log_imp) × total_rain_mm
pipe_drain_cap = expm1(path_cap) + 0.1
conv_def      = log1p(inflow_load / pipe_drain_cap)                        # conveyance deficit
dep_escape    = dep_depth / (max(0.005, |max_in_grade|) + 0.01)
```

Feeds `x_full` = [26 physical, `deg_diff`, `total_rain_mm`, `dyn_sat`, `true_ponding`, `conv_def`, `dep_escape`] = **32 node features**. `intensity_mmhr` → col 13, `duration_min` → col 14 are stamped live; col 25 intensity-scaled, col 27 = total rain, col 28/29/30 = `dyn_sat`/`true_ponding`/`conv_def`.

2 edge features (`edge_c=2`): col 0 = conduit length, col 1 = conduit **grade** (slope).

The gravity gate (§4) reads `ea[:, 1:2]`.

3.2 Dataset

- `expanded_master_physics_dataset.pt` — **480 graphs, 2,350,544 nodes**; 8 storm intensities × durations; 16 global catchments (76,316 *independently audited* physical nodes); ground truth from **EPA SWMM 5.2** dynamic-wave 1D/2D solutions.
- Valley-sink fixes applied in `build_expanded_physics_dataset.py`:
- `x[:,23] = where(rel_drop≥0.50 & dep_depth≥0.05, dep_depth, 0)`
- `x[:,6] = where(rel_drop≥0.50 & dep_depth≥0.08, 1.0, 0.0)`
- ponding index `x[:,29] = log1p(x[:,23] · total_rain / (max(0.2, out_deg) + 0.3))`
- `elev_range` (true catchment relief in m) baked onto graphs for the WSE backwater envelope.
- **Train/val split** (`train()`): val regions = {'hsr', 'bellandur', 'london', 'newyork'}.

3.3 Normalization & target

```
x_mean/x_std: standardize per column on TRAIN graphs only (zero leakage)
e_mean/e_std: standardize edge_attr
y_log = log1p(clamp(y, 0))
yl_mean, yl_std = mean/std of y_log
y_norm = (y_log - yl_mean) / yl_std      # regressor target
```

Decode: `p_lin = clamp(expm1(raw_depth · yl_std + yl_mean), 0, 3)`.

The `log1p` domain is the reason the loss is Huber-in-log-space: log-domain errors on deep basins amplify into meter-scale spikes after `expm1` inversion (documented in `train..._v5_0.py:41–47`).

4. Model Architecture (Verbatim)

4.1 GravityGINEConv (the v5 novelty) — production_v4.py:192–210

```
class GravityGINEConv(nn.Module):
    def __init__(self, in_c, out_c, edge_c=2):
        super().__init__()
        self.conv = GINEConv(nn.Sequential(
            nn.Linear(in_c, out_c),
            nn.LayerNorm(out_c),
            nn.LeakyReLU(0.1),
            nn.Linear(out_c, out_c)), edge_dim=edge_c)
        self.ln = nn.LayerNorm(out_c)

    def forward(self, h, ei, ea):
        grade = ea[:, 1:2]
        gravity_gate = torch.sigmoid(1.0 - 5.0 * F.relu(grade)) # downhill=full, uphill attenuated
        ea_gated = ea * gravity_gate
        return F.leaky_relu(self.ln(self.conv(h, ei, ea_gated)), 0.1)
```

Physics meaning: **downhill edges (grade ≤ 0) pass flow unattenuated** ($\text{sig}(1) \approx 0.731 \rightarrow \text{gate} \approx 1$); **steep uphill edges are exponentially attenuated** ($\text{grade}=0.4 \rightarrow \text{sig}(-1)=0.269$; $\text{grade}=1 \rightarrow \text{sig}(-4)=0.018$) — water does not flow up conduits.

4.2 HydroGINE_v5 backbone — production_v4.py:213–268

```
6 × GravityGINEConv (in_c=32 → 128, then 128 → 128)
residual: h = h_next + 0.3 · h (for i > 0; preserves topology – NO pooling)
mid_h captured at layer i == n_layers//2 (multi-scale skip)
cat = concat([h, mid_h, x]) → cat_dim = 128·2 + 32 = 288

Classifier head cls: Linear(288,128)→LN→LReLU(0.1)→Dropout(0.05)→Linear(128,64)→LN→LReLU→Linear(64,1)
FiLM generator: Linear(1,64)→LReLU→Linear(64, 2·cat_dim=576) [LAST LAYER ZERO-INIT]
Regressor head reg: Linear(288,256)→LN→LReLU→Dropout(0.05)→Linear(256,128)→LN→LReLU→Linear(128,1)
```

Forward:

```
def forward(self, x, ei, ea):
    h = x; mid_h = None
    for i, conv in enumerate(self.convs):
        h_next = conv(h, ei, ea)
        h = h_next + 0.3*h if (i > 0 and h.shape == h_next.shape) else h_next
        if i == len(self.convs)//2: mid_h = h
    cat = torch.cat([h, mid_h, x], dim=-1)
    cls_logits = self.cls(cat)
    prob = torch.sigmoid(cls_logits)
    film = self.film_gen(prob)
    gamma, beta = torch.chunk(film, 2, dim=-1)
    h_cond = cat * (1.0 + gamma) + beta # FiLM modulation, identity at init
    raw_depth = self.reg(h_cond)
    return cls_logits, raw_depth
```

Design notes:

- **Zero-initialized FiLM** → identity modulation at start, so depth regressor trains clean then learns hazard×depth cross-task conditioning later (prevents zero-gradient collapse).
- **Margin focal classifier** head drives the prob that (a) FiLM-modulates depth and (b) feeds the production confidence gate.

4.3 Hydrological rationale (paper narrative)

10-m SRTM/Copernicus DEM + OSM street centerlines → directed graph $G=(V,E)$; Kirpich time of concentration $T_c = 0.0195 L^{0.77} S^{-0.385}$; TWI, sink-depression depth, drainage capacity ratio all pre-baked as features.

5. Final Loss (v5.11, Verbatim)

train_hydro_gine_v5_0.py lines 327-445. Config constants:

```
EPOCHS          = 300          LR = 3e-4          WEIGHT_DECAY = 1e-4
HIDDEN_DIM      = 128          N_LAYERS = 6          N_CHUNK = 20000 (was 25000; RTX 4060 TDR fix)
SEED            = 42
LOSS_ASSYM_UP   = 1.5          # over-pred on DRY cells ( $y \leq 0.15$ )
LOSS_ASSYM_UP_SHALLOW = 1.5  # over-pred in advisory band ( $0.15 < y \leq 0.50$ )
LOSS_ASSYM_UNDER = 2.0        # under-pred on FLOODED cells ( $y > 0.15$ )
DEEP_W_STRENGTH = 3.0         DEEP_W_THRESH = 1.5     DEEP_W_RANGE = 2.5     DEEP_W_POWER = 2.0
MATCH_BAND_M    = 0.12        MATCH_HINGE_STRENGTH = 0.0
DEEP_TARGET_W   = 2.2         DEEP_TARGET_W_MID = 1.3
SELECT_RAIL_MATCH = True      EQUAL_WEIGHT_REGIONS = True
MARGIN_FOCAL    = MarginFocalLoss(gamma=2.0, alpha=0.35, margin=0.02)
```

```
# 1. Margin Focal Loss on hazard (binary  $y \geq 0.15$ )
loss_cls = focal_loss(cls_logits, y_hazard)

# 2. Zero-Bias Asymmetric Huber (log-domain),  $\delta = 0.10$ 
diff = depth_pred_log - y_norm
abs_diff = torch.abs(diff)
huber = torch.where(abs_diff < delta, 0.5*diff**2, delta*(abs_diff - 0.5*delta))

over_mask = (diff > 0.0) & (y_true > 0.15) & (y_true <= 0.50)
asym = torch.where(
    (diff > 0.0) & (y_true <= 0.15), LOSS_ASSYM_UP,          # 1.5 dry over
    torch.where(over_mask, LOSS_ASSYM_UP_SHALLOW,          # 1.5 advisory o
    torch.where((diff < 0.0) & (y_true > 0.15), LOSS_ASSYM_UNDER, 1.0))) # 2.0 flooded un
huber = huber * asym

# 3. Node weights
sink_d = batch.x[:, 23]; dep_d = batch.x[:, 16]
is_dry_pavement = (y_true <= 0.03) & (sink_d < 0.03)
w = torch.ones_like(y_true)
w = torch.where(y_true >= 0.30, 3.0, w) # critical
w = torch.where((y_true >= 0.15) & (y_true < 0.30), 2.0, w) # advisory
w = torch.where(is_dry_pavement, 3.0, w) # dry pavement hard-negative
w = torch.where(y_true >= 0.50, w * DEEP_TARGET_W, w) # 2.2 deep-tail
w = torch.where((y_true >= 0.30) & (y_true < 0.50), w * DEEP_TARGET_W_MID, w) # 1.3
deep_w = 1.0 + DEEP_W_STRENGTH * clamp((dep_d - DEEP_W_THRESH)/DEEP_W_RANGE, 0, 1)**DEEP_W_POWER
w = w * deep_w
loss_reg = (w * huber).mean()

# 4. Flood-gated match hinge (OFF in final; MATCH_HINGE_STRENGTH = 0.0)
pred_lin = clamp(expm1(depth_pred_log * yl_std + yl_mean), 0)
res_lin = pred_lin - y_true
hinge_mask = (y_true > 0.15)
match_hinge = relu(|res_lin| - MATCH_BAND_M)
loss_match = MATCH_HINGE_STRENGTH * sum(hinge_mask * w * match_hinge) / n_flooded

total = loss_cls + 1.5*loss_reg + 0.8*loss_match
```

Optimization: AdamW(lr=3e-4, wd=1e-4), CosineAnnealingLR → eta_min=1e-5, grad-clip 1.0, mini-epochs of 20k-node chunks, warm-start param-transfer from hydro_gine_v5_10_model.pt, **early stop** after 9 no-improve validation evals.

5.1 MarginFocalLoss (exact, reused in v4/v5)

```
class MarginFocalLoss(nn.Module):
    def forward(self, logits, targets):
        probs = torch.sigmoid(logits).clamp(1e-6, 1 - 1e-6)
        targets_m = torch.where(targets == 1.0, 1.0 - self.margin, self.margin)
        p_t = torch.where(targets == 1.0, probs, 1.0 - probs)
        alpha_t = torch.where(targets == 1.0, self.alpha, 1.0 - self.alpha)
        focal_weight = alpha_t * (1.0 - p_t)**self.gamma
        return (focal_weight * BCELogits(logits, targets_m)).mean()
```

6. The Loss Family — Every Formula Ever Used (Precise)

6.1 Phase 0 — PINN prototypes

train_pinn_gnn.py (physics_informed_loss, $\lambda=0.5$) — GATv2 8-feature prototype; replaced.

train_ultra_pinn.py **ultra_pinn_loss** (exact):

```
huber_loss      = huber(pred, target, 0.15)
mse_loss        = F.mse_loss(pred, target)
focal_depth     = mean((target > 0.5) * (pred - target)**2)      # outfall high-depth focal
non_neg_penalty = mean(rel(-pred)**2)
L = huber + 0.5*mse + 1.2*focal_depth + 0.2*non_neg_penalty
```

AdamW lr=3e-3 wd=1e-5, ReduceLROnPlateau(factor=0.5, patience=50), 1000 epochs.

master_pinn_engine.py **compute_master_physics_loss** (full Saint-Venant + Bernoulli; verbatim in file):

```
high_depth_mask = (target > 0.4)
focal_loss      = mean(high_depth_mask * (pred - target)**2)
non_neg         = mean(rel(-pred)**2) ; surcharge = mean(rel(pred - 3.0)**2)
velocity        = (1/n) * sqrt(|grade|) ; flow_q = depth * velocity
node_inflow/outflow via index_add over edges
mass            = mean(clamp((inflow - outflow)**2, max=25))
S_f             = n**2 * v**2 / depth**(4/3)                      # Manning friction slope
momentum        = mean(clamp(((d_dst-d_src) + grade*L - clamp(S_f,2)*L)**2, max=25))
E_head          = elev + depth + v**2/2g                         # Bernoulli energy head
energy          = mean(clamp((E_src - E_dst - S_f*L)**2, max=25))
L = huber + 0.4*mse + 1.0*focal + 0.2*non_neg + 0.05*surcharge + 0.08*mass + 0.04*momentum + 0.04*ene
```

active_pinn_pipeline.py **compute_saint_venant_pinn_loss** ($\lambda_{\text{mass}}=0.05$, $\lambda_{\text{momentum}}=0.02$, $\lambda_{\text{phys}}=0.2$):

```
L = mse + 0.2*non_neg + 0.05*surcharge + 0.05*mass + 0.02*momentum
```

- active Monte-Carlo dropout sampling (20 samples) boosting loss 0.2× on top-20%-variance nodes.

train_dual_stream_hydro_gnn.py (multi-task "hurdle gate"):

```
p_gate = sigmoid(gate_logit)
focal_weight = gate_target*(1-p_gate)**1.5*2.0 + (1-gate_target)*p_gate**1.5
gate_loss = mean(focal_weight * BCE(gate_logit, gate_target))
depth_loss = sum(wet_mask*crit_scale*SmoothL1(pred, target,  $\beta=0.02$ )) / (wet_mask.sum()+1e-6)
L = depth_loss + 3.0*gate_loss + safe_penalty
```

6.2 Phase 1 — st_gnn/ research line (iter 1–18)

Common: x.zscore, log1p(y) target, chunk streaming, grad clip 1.0, AdamW + Cosine.

- **iter1/2**: $L_{\text{log}} = \text{mean}(w_{\text{node}} \cdot (\text{out}-\text{ty})^2) + 0.5 \cdot \text{mean}(w_{\text{node}} \cdot w_{\text{asym}} \cdot d^2)$, $w_{\text{asym}} = 2.5$ under / 1.0

- **iter6–11 (TwoHeadGINE)**: $L_{\text{reg}} + 0.5 \cdot L_{\text{asym}} + 0.7 \cdot L_{\text{bce}} + 0.3 \cdot L_{\text{dice}}$, hilly_w=2.0 on elev₂>1.3

- **iter12 (focal):** $\alpha_t = t \cdot 0.25 + (1-t) \cdot 0.75$, $0.7 \cdot L_{\text{focal}} + 0.3 \cdot L_{\text{dice}}$
- **iter13–17 (FiLMGINE):** homoscedastic $\text{prec_cls} \cdot L_{\text{cls}} + 0.5 \cdot s_{\text{cls}} + \text{prec_reg} \cdot L_{\text{reg}} + 0.5 \cdot s_{\text{reg}}$, zero-init identity FiLM; iter17 warm-started from contrastive encoder
- **iter18 (FiLMGINE_SAGPool):** $\text{MarginFocal}(\gamma=2, \alpha=0.25, m=0.03) + 0.3 \text{ soft-dice} + 0.5 \text{ asym-MSE}$ pooled to coarse nodes — **collapsed** (§11).

6.3 Phase 2 — v4 lineage

Common scaffold: $\text{in_c}=32$, $\text{hidden} \in \{96, 128\}$, $\text{n_layers}=6$, residual $0.3\times$, skip $\text{cat}=[h, \text{mid_h}, x]$, $\text{cat_dim}=\text{hidden} \cdot 2 + 32$, FiLM zero-init, bi-head.

Ver	hidden	epochs	lr/wd	chunk	loss delta
v4	128	350	3e-3/1e-5	100k	asym MSE + BCE + soft-dice
v4.1	96	160	7e-4/1e-4	20k	LeakyReLU encoder; MarginFocal(2.0, 0.35, 0.02)
v4.2	128	600	1.2e-3/2e-5	100k	cosine restarts; same MarginFocal
v4.3	96	250	2.5e-4/1e-5	25k	homoscedastic $\log_{\text{var_cls}}/\log_{\text{var_reg}}$ weighting; warm-start v4.1

The v4 "clay-and-pavement" lesson reframed into cannot-v5 two-task **classify + regress + FiLM-gate**.

7. Production Physical Rails (Verbatim)

`apply_physics_rails(x_full, ei, p_lin, p_prob)` — the "Universal Physical Continuity Bounding"
post-processing. Feature cols used: $\text{rel_drop}=0$, $\text{in_d}=3$, $\text{out_d}=4$, $\text{accum_s}=5$, $\text{is_sink}=6$, $\text{sag_idx}=8$, $\text{dep_d}=16$, $\text{sink_d}=23$, $\text{total_r}=27$, $\text{conv_def}=30$.

```

is_choked_surcharge = (conv_def >= 0.7) & ((sag_idx > 0.01) | (dep_d > 0.02) | (out_d < in_d))
is_deep_sink        = is_sink & (dep_d >= 0.20)
is_valley_depression= dep_d >= 0.20
is_convergent_sag   = (in_d > out_d) | (sag_idx >= 0.03)
is_ridge_crest      = (rel_drop < 0.25) & (sink_d < 0.03) & (dep_d < 0.03)
is_free_drain_slope = (sink_d < 0.02) & (dep_d < 0.02) & (out_d >= 2) & ~choked
is_steep_ridge      = (rel_drop < 0.20) & (|slope| > 0.06) & (accum_s < 0.5) & (sink_d < 0.01) & ~cho

tau = 0.15 if (deep_sink|valley) else
      0.25 if (choked|sag) else
      0.75 if (ridge|free_drain|steep) else 0.35
conf_gate = 1 / (1 + exp(-6*(p_prob - tau)))          # steep logistic confidence gate

mass_bound: choked → 3.0
              deep_sink|valley → clip(dep_d*1.5 + 0.3, 1, 3)
              ridge|steep → 0.01 ; free_drain → 0.04 (if rain≤50 mm) else 0.10
              (p_prob ≥ 0.65 & sag) → max(0.50, sink_d*2 + 0.25)
              (p_prob ≥ 0.65)      → max(0.50, sink_d*1.5 + 0.20)
              else                  → max(0.15, sink_d*1.5 + 0.08)

p_final = min(p_lin * conf_gate, mass_bound)
p_final = 0 where (flat_dry | steep_ridge); 0 where < 0.02; cap 3.0

# WSE backwater envelope (relief_m = elev_range, default 15.0)
elevs    = -max(max(dep_d, sink_d), rel_drop*relief_m)
wse      = elevs + p_final
backwater = relu(wse[src] - elevs[dst]) (both directions; max-scatter per node)
is_protected = deep_sink | valley | choked
p_final[~protected] = min(p_final[~protected],
                          max(max_backwater[~protected], own_storage[~protected]))

```

This single transformer converts **raw ~47 %** accuracy into deploy-time **~83 %**; checkpoint selection data accordingly uses **post-rail** metrics (`SELECT_RAIL_MATCH=True`).

8. Final Ensemble (Verbatim)

EnsembleFloodPredictorV4 (production_v4.py:443–527). Key logic:

```
model_paths = ("hydro_gine_v5_10_model.pt",      # w=0.80
               "hydro_gine_v5_11_model.pt",      # w=0.30
               "hydro_gine_v5_0_model.pt",        # w=0.05
               "hydro_gine_v5_bangalore_opt.pt")  # w=0.10
weights      = (0.80, 0.30, 0.05, 0.10)         # re-normalized by w_sum in __init__
...
def predict(self, graph_or_batch, intensity_mmhr, duration_min=60.0):
    raw_list, prob_list = [], []
    for m, w in zip(self.members, self.weights):
        p_rail, p_lin, p_prob = m.predict(...)
        raw_list.append(p_lin.astype(np.float64) * w)
        prob_list.append(p_prob.astype(np.float64) * w)
    p_lin_avg = np.sum(stack(raw_list), axis=0)
    p_prob_avg = np.sum(stack(prob_list), axis=0)
    p_final = apply_physics_rails(x_full, ei, p_lin_avg, p_prob_avg, elev_range=relief_m)
    return p_final, p_lin_avg, p_prob_avg
```

Critical design: **average raw pre-rail p_lin/p_prob , apply rails ONCE** — rails applied per-member would double-count the confidence gates and mass bounds. Missing members are skipped; an empty ensemble falls back to v5.10 then bottleneck_opt with unity weight.

9. Full Version History From Git (Including Regressions)

Commit map, newest → oldest (git log --oneline, subset):

Commit	Artifact	Change	Headline
3f4dca8	final tree	cleanup for competition transfer	—
a18ac71	FINAL	weighted 4-model ensemble	HSR 82.2, Bellandur 72.0, NSE 0.9098
b806d3d	v5.11	deep reweight 2.2/1.3	87.11 % (regression!) ← worse than v5.10
2eb0a38	v5.10	deep reweight 1.6/1.2 (sweet spot)	88.14 % leader
b4ad8d2	Ens v5.6+v5.8	weighted raw-depth ensemble	85.88 %, NSE 0.8911
1367d46	v5.9	deep reweight 3.0/1.4	NSE 0.8942 best-ever quality, match 84.5 % ← regression vs v5.8
0f90840	v5.8	EQUAL_WEIGHT_REGIONS	84.8 % leader , OVER 977
ee57de0	v5.7	shallow-band over-penalty	84.07 % ← regression vs v5.6
240d86c/b6a2ee4	v5.6	post-rail checkpoint selection	84.5 %, NSE 0.8814
3f2519c	v5.3–5.5	match-hinge/flood-gated experiments	all worse than v5.2
291be43	v5.2	scenario-conditional asym (under 2× flooded)	83.3 %, MAE 2.97cm, NSE 0.866
16f5130	v5.0	zero-bias asymmetric Huber	82.0 %, OVER 3141→692
b53f51d	v5 base	HydroGINE-v5.0 + master dataset	77.8 %
(pre-v5)	v4→v4.3	MarginFocal + homoscedastic	predecessor line
(pre-v4)	st_gnn iter1–18	two-task GINE + FiLM + contrastive	iter18 SAGPool collapsed
(Phase0)	PINN prototypes	GATv2 + Saint-Venant	abandoned

9.1 v5 hyperparameter sweep (the core story)

Ver	EPOCHS	LR	N_CHUNK	UP	UP_SH	UNDER	DEEP_W	DEEP_MID	HINGE	RAIL	EQREG	Match	NSE
base	200	3e-4	25k	—	—	2.5×MSE	—	—	—	✗	✗	77.8	0.7764
v5.0	200	3e-4	25k	1.5	—	—	—	—	—	✗	✗	82.0	0.8465
v5.2	220	3e-4	25k	1.5	—	2.0	—	—	—	✗	✗	83.3	0.8659
v5.4	220	3e-4	25k	1.5	—	2.0	—	—	2.0	✗	✗	81.7	0.8606
v5.5	220	3e-4	25k	1.5	—	2.0	—	—	0.0	✗	✗	80.5	0.8647
v5.6	300	3e-4	25k	1.5	—	2.0	—	—	0.0	✓	✗	84.5	0.8814
v5.7	300	3e-4	25k	1.5	1.5	2.0	—	—	0.0	✓	✗	84.1	0.8745
v5.8	300	3e-4	20k	1.5	1.5	2.0	0	—	0.0	✓	✓	84.8	0.8779
v5.9	300	3e-4	20k	1.5	1.5	2.0	3.0	1.4	0.0	✓	✓	84.5	0.8942
v5.10	300	3e-4	20k	1.5	1.5	2.0	1.6	1.2	0.0	✓	✓	88.1	0.9060
v5.11	300	3e-4	20k	1.5	1.5	2.0	2.2	1.3	0.0	✓	✓	87.1	~0.900

(v5.1 = global match-hinge → OVER exploded, removed. v5.3 = hinge-strength sweep.)

10. Benchmark Tables

10.1 Full 10-version table (benchmark_all_versions.json)

Model	Match %	NSE	GlbMAE cm	HazMAE cm	Risk	OVER	UNDER
v5.6	84.54	0.8814	2.80	10.30	17604	1847	875
v5.7	84.07	0.8745	2.88	10.83	17093	1733	990
v5.2	83.30	0.8659	2.97	11.18	16986	1403	1433
v5.0	81.96	0.8465	3.14	12.53	16373	692	2262
v5.4	81.67	0.8606	3.11	11.10	17685	1975	1266
v5.5	80.55	0.8647	3.37	10.77	20248	2301	1638
v5 Base	77.79	0.7764	3.84	15.15	16571	1227	2453
v5 Bottleneck	77.15	0.8284	5.15	12.70	22753	3141	2057
v5 Bengaluru	77.02	0.8292	4.53	13.08	20463	2182	2521
v5 Finetuned	76.70	0.8187	5.71	13.13	22776	3286	2020

10.2 Per-city progression (HSR + Bellandur — the local priority)

Version	HSR %	Bellandur %	Whitefield %	ECity %	Koramangala %
v5.6	76.79	68.62	83.48	83.27	73.31
v5.8	78.31	70.39	83.81	81.95	74.70
v5.6+v5.8 ens	78.24	70.54	85.98	84.29	76.01
Final 4-model ens	82.2	72.0	—	—	—

10.3 Phase-2 competition benchmark (`competition_benchmark_verified.json`) — final system

Goal 1 — hotspot match (± 15 cm, 30 nodes/city):

- Global hotspot match rate **94.79 %**, worst-case district **86.67 %**, hazard recall **99.58 %**
- Best cities: Tokyo/London/Bangkok 100.0 %; Bellandur 100.0 %, HSR 93.3 %, Berlin 93.3 %, Delhi 93.3 %
- Koramangala & Hong Kong at 86.67 % (worst-case)

Goal 2 — flooded-node detection (100 mm/hr, regenerated verified)

- Flooded nodes 20,966 (27.47 % gated); recall **93.37 %**, precision **89.43 %**, **F1 0.9136**; flooded-binary MAE 7.81 cm.
- HSR: recall 92.16 %, precision 73.44 %, F1 0.817, flooded MAE 5.29 cm.

Goal 3 — NSE / hazard depth MAE / mass continuity: 0.9128 catchment NSE / 0.8765 flooded NSE, 8.65 cm hazard MAE, 8.61 % mass-continuity error.

Note: the pre-regeneration values (hotspot 93.13 %, F1 0.803, MAE 14.52 cm, NSE 0.8338) were superseded when `competition_benchmark_verified.json` was rebuilt live; the numbers above are the canonical, reproducible output. `round2_live_benchmark_*.json` in §10.4 is an earlier run and is retained as a historical record only.

10.4 Round-2 live (50 mm/hr vs 100 mm/hr, top-30 audit)

- 50 mm/hr: F1 range 0.731 (NYC)–0.904 (Bangkok); HSR F1 0.845 / top-30 96.7 %; worst MAE 7.43 cm (Hong Kong @50).
- 100 mm/hr: per-city flood F1 0.683 (Koramangala)–0.876 (Tokyo); each city keeps $\geq 26/30$ top-30 hotspots matched.
- Live-match speedup: $\text{swmm_speedup} = 3500.0 * (15.0 / \text{avg_city_latency_ms}) \rightarrow v5.6 = 5004\times$, $v5.8 = 6471\times$, ens $v5.6+v5.8 = 1966\times$ (heavier, 26.7 ms/city).

11. Empirical Validation Data Provenance & Source Verification

This section documents the exact origin of every piece of real-world data used to validate UrbanFLOW on Bengaluru, so that no claim in this paper rests on an unidentified or synthetic source. It exists because the validation must be **falsifiable**: a reader must be able to open each cited news report and confirm that the named location, date, and rainfall figure are real.

11.1 Governing principle: no fabricated incidents

The empirical validation set contains **no invented locations, dates, or rainfall values**. Every incident corresponds to a place explicitly named in a published news report, geocoded from OpenStreetMap (Nominatim). Where a report gives a corridor rather than a point (e.g. "Sarjapur Road"), the geocode represents the named corridor and this is stated in the entry's `coordinate_note`. The dataset asserts **no flood depths, no incident counts, and no control-room records** that are not present in the cited sources.

11.2 Source dataset and schema

- **File:** `data/real_bengaluru_oct2024_incidents.json` (schema_version 1.0).
- **Scope:** flood-affected locations inside the five UrbanFLOW Bengaluru catchments (HSR, Koramangala, Bellandur, Whitefield, Electronic City).
- **Structure:** three `events` (with per-event rainfall and full article lists) + thirteen `incidents` (each with `region`, linked `event_ids`, `location`, `lat/lon`, `coordinate_note`, `description`, and one or more `sources` URLs).
- **Origin of the file:** hand-assembled from the cited reports and Nominatim geocoding; it is the source of truth consumed live by `validate_real_world_incidents.py` and by the portal endpoint `/api/historical-validation` (`app.py:1352`). It is **not** model output.

11.3 The three October 2024 rain events (rainfall attribution)

Event ID	Dates	Rainfall figure and its source	What the sources document	Publishers
bengaluru_oct15_2024	2024-10-14 → 15	65 mm city-wide, 8 am–8 pm Oct 15; 228 % above seasonal average (BBMP / NIE)	Waterlogging at 142 places ; 52 areas and 142 houses flooded; 34 places declared flood-prone by BBMP	The Hindu, New Indian Express, Moneycontrol, Times of India
bengaluru_oct20_2024	2024-10-19 → 20	Kengeri 141 mm / 24 h (BBMP via TOI); Jnana Bharathi, RR Nagar, Nayandahalli 106 mm ; IMD Bengaluru city station 19.7 mm	Hosur Road (Roopena Agrahara) and Silk Board Junction flooded; Varthur–Gunjur Rd, ITPL Rd, Mysuru Rd corridor waterlogged till mid-morning	Times of India, The Hindu, New Indian Express
bengaluru_oct22_2024	2024-10-21 → 22	IMD GKVK 186.2 mm / 24 h — highest single-day October rainfall in 27 years (previous record 178.9 mm on 1997-10-01); BBMP Yelahanka zone 157 mm / 6 h ; city total 264.5 mm to 08:30 Oct 22	Two lakes breached (Kogilu, Doddabommasandra); NDRF/SDRF boats at Kendriya Vihar; ~100 lakes overflowing; ORR/IT-corridor flooding	Indian Express, New Indian Express, Times of India, Times Now

11.4 Incident-level provenance map (all 13 incidents)

Each bullet gives the incident ID, its catchment, the event(s) it belongs to, and the exact article URL that names the location. Full titles/dates are in the JSON `sources` blocks.

- 1. REAL-KOR-01-SILK-BOARD** — Koramangala, events oct15+oct20 — "Silk Board Junction and Hosur Road" — The Hindu [.../bengaluru-weather-heavy-rains-lead-to-waterlogging-and-traffic-chaos.../article68755266.ece](#); The Hindu [.../early-morning-rains-flood-bengaluru-roads-and-apartments/article68775016.ece](#).
- 2. REAL-KOR-02-MADIWALA** — Koramangala, event oct15 — "Madiwala junction (Hosur Road)" — NIE [.../2024/Oct/16/traffic-goes-haywire-on-major-roads](#).
- 3. REAL-HSR-01-ROOPENA-AGRAHARA** — HSR, event oct20 — "Hosur Road at Roopena Agrahara" — TOI [.../articleshow/114404508.cms](#); NIE [.../2024/Oct/21/rain-woes-continue-to-haunt-bengaluru](#).
- 4. REAL-HSR-02-HSR-LAYOUT** — HSR, event oct22 — "HSR Layout (Sector 2 / 27th Main)" — NIE [.../2024/Oct/23/north-bengaluru-crumbles-under-cloudburst](#).
- 5. REAL-HSR-03-SARJAPUR-ROAD** — HSR, events oct15+oct22 — "Sarjapur Road (Wipro / RGA Tech Park)" — Moneycontrol [.../manyata-tech-park-in-bengaluru-flooded...-12842550.html](#); Indian Express [.../rainfall-north-bengaluru-breach-two-lakes-yelahanka-zone-9634106/](#).
- 6. REAL-HSR-04-BOMMANAHALLI** — HSR, events oct15+oct20 — "Bommanahalli junction (Hosur Road)" — NIE [.../2024/Oct/16/traffic-goes-haywire-on-major-roads](#); The Hindu [.../article68775016.ece](#).
- 7. REAL-BEL-01-ORR-IBLUR** — Bellandur, event oct22 — "ORR service road between Iblur and Marathahalli" — Indian Express [.../rainfall-north-bengaluru-breach-two-lakes-yelahanka-zone-9634106/](#).
- 8. REAL-BEL-02-ECOSPACE** — Bellandur, event oct22 — "Ecospace junction (ORR)" — Indian Express [.../rainfall-north-bengaluru-breach-two-lakes-yelahanka-zone-9634106/](#).
- 9. REAL-BEL-03-DEVARABEESANAHALLI** — Bellandur, event oct22 — "Devarabeesanahalli (ORR)" — NIE [.../2024/Oct/23/north-bengaluru-crumbles-under-cloudburst](#).
- 10. REAL-BEL-04-BELLANDUR-LAKE** — Bellandur, event oct22 — "Bellandur Lake overflow / ORR service roads" — Indian Express [.../rainfall-north-bengaluru-breach-two-lakes-yelahanka-zone-9634106/](#); NIE [.../2024/Oct/23/north-bengaluru-crumbles-under-cloudburst](#).
- 11. REAL-WHI-01-ITPL-ROAD** — Whitefield, event oct20 — "ITPL Road / International Tech Park Bengaluru" — TOI [.../articleshow/114404508.cms](#).
- 12. REAL-WHI-02-HOPEFARM** — Whitefield, event oct20 — "Hope Farm junction (Whitefield)" — TOI [.../articleshow/114404508.cms](#).
- 13. REAL-ECI-01-ELECTRONIC-CITY** — Electronic City, events oct15+oct20 — "Electronic City Phase 1 (Hosur Road)" — Moneycontrol [.../manyata-tech-park-in-bengaluru-flooded...-12842550.html](#); The Hindu [.../article68775016.ece](#).

Event-level articles not tied to a single incident (full URLs in the JSON): Indian Express

[.../bengaluru-rain-building-collapse-schools-closed-flooding-overflowing-lakes-9633673/](#), New Indian Express

.../2024/Oct/22/cloud-burst-hits-bengaluru-yelahanka-zone-10-layouts-flooded-4000-residents-impacted, and Times Now .../bengaluru-rains-break-27-year-old-october-record...-114455130.

11.5 Coordinate provenance

All thirteen coordinates were geocoded with **OpenStreetMap Nominatim** (e.g. "Silk Board Junction Bengaluru", "EcoSpace Bengaluru", "Iblur Junction Bengaluru"). Each incident stores a `coordinate_note` recording the query and, where the source names a corridor rather than a point, states that the point is a representative geocode. Coordinates were not chosen to improve the model's hit rate.

11.6 How this dataset is used (validation methodology)

`validate_real_world_incidents.py` starts the live server, loads these incidents, and for each one runs the shipped ensemble (HydroGINE-v5: v5.10 + v5.11 + v5.0 + BangaloreOpt) at an assumed short-duration peak intensity of **100 mm/hr for 60 min** (the published Oct 2024 figures are 6 h / 24 h accumulations and cannot be fed to a minute-scale urban-inundation model directly). An incident is **captured** iff a predicted hazard node ($\text{gnn_depth} \geq 0.15 \text{ m}$) lies within **50 m** of the geocoded location. The canonical live result is **5 / 13 captured (38.5 %)** (`real_world_validation_result.json`); all five captures are in the Koramangala–Bellandur corridor (KOR-01 33.7 m, KOR-02 10.2 m, BEL-01 29.8 m, BEL-02 27.8 m, BEL-03 26.8 m). The misses are **spatial**, not intensity-driven: the model's hazard nodes do not reach those points even at the high assumed rate, and the paper reports this miss rate rather than tuning the assumed intensity upward to hide it. `historical_validation_result.json` is a stale, malformed artifact and is **not** authoritative.

11.7 Real vs. synthetic — the explicit provenance boundary

To prevent conflation, the two data classes are separated here:

- **Real (news-documented):** the incident *locations, dates, event linkage*, and the *rainfall figures* in §11.3/§11.4 above. These come only from the cited reports.
- **Synthetic (model-generated):** the SWMM reference depths that the GNN regresses against. The 50 mm/hr references are stored targets in `expanded_master_physics_dataset.pt` (`region_graphs_50[r].y`) / `swmm_groundtruth_targets.csv`, not re-run during validation.
- **Important caveat:** the 100 mm/hr "SWMM" reference in the competition benchmark is **not** a genuine SWMM dynamic-wave re-run. It is a heuristic scaling of the 50 mm/hr baseline inside `app.py` (lines 919–925: `base_d*(1+0.55*(ratio-1)) + 0.082*(ratio-1)`), and the GNN output is post-adjusted at

50 mm/hr (`app.py:927: pred*1.04+0.02`). This is a documented approximation, not a physics run.

- **Also hardcoded:** the goal-4 speedup/latency constants are literal values in `run_competition_master_benchmark.py` (lines 194–219), not measured in-run. The models were **not** retrained for any of the reported numbers.

11.8 Verification log

During preparation of this revision the three events and the incident attributions were checked against the live articles. Confirmed: the 65 mm city-wide total, 52 areas / 142 houses flooded, and 142 waterlogging points (Oct 15); Kengeri 141 mm and the Hosur Road / Silk Board flooding (Oct 19–20); and the IMD GKVK 186.2 mm 27-year October record with the Yelahanka cloudburst and lake breaches (Oct 21–22). The Indian Express Yelahanka-zone report names Ecospace junction, the Iblur–Marathahalli ORR service roads, Bellandur Lake overflow, and Sarjapur Road; the NIE "North Bengaluru crumbles under cloudburst" report names Devarabeesanahalli, HSR Layout, Bellandur, Sarjapur and Karthik Nagar. Every incident citation in §11.4 therefore matches a location actually named in its cited source.

12. Key Lessons / Why Some Versions Became Worse

1. **Global match-hinge (v5.1) → removed.** A global linear hinge that penalized `|residual - band|` on *all* nodes inflated OVER explosively; the fix (v5.2) gated it to flooded cells only (`hinge_mask = y > 0.15`), but the final config disables it entirely (`STRENGTH = 0.0`).
 2. **v5.4 < v5.2 (81.67 % vs 83.30 %):** activating the hinge with strength 2.0 *with* dry-gate still hurt; v5.5 confirms hinge→0 (80.55 %, even lower due to the flood-gated-interim variant). Lesson: **hinge terms map to the match metric only in expectation; they trade off regression quality.**
 3. **v5.7 < v5.6 (84.07 % vs 84.54 %):** extending the 1.5× over-penalty into the shallow advisory band (0.15–0.50) over-punished legitimate over-predictions and pressed down edges that v5.6 matched — flip side of the same lesson.
 4. **v5.9 vs v5.8 (match 84.5 % vs 84.8 %) despite best-ever NSE 0.8942:** the deep-tail hammer `DEEP_TARGET_W=3.0` recovered deep pools but **pushed extreme dry bowls up** (OVER 1623), breaking the *match* (depth-band) metric while improving *volume* (NSE). **NSE and live-match are different objectives** — a 3× tail weight is not free.
 5. **v5.11 < v5.10 (87.11 % vs 88.14 %):** overshooting `DEEP_TARGET_W` from 1.6 → 2.2 re-created dry-bowl OVERs; the optimum is exactly at **1.6/1.2**. v5.11 is kept in the ensemble **only** for deep-tail recovery at low weight (0.30), so its regression datum actually contributes complementarity instead of harm.
 6. **Pre-rail vs post-rail confusion (≤v5.5):** without `SELECT_RAIL_MATCH`, checkpoints were selected on raw accuracy (~47 %), systematically saving models the rails would later crush. Moving selection to **post-rail** (v5.6) was a +~7 pp jump.
 7. **Node-weighted vs region-equal aggregation (v5.7→v5.8):** the node-weighted val average was dominated by London + NYC (largest graphs), silently trading away Bengaluru districts (v5.6: HSR 84.7→76.8, Bellandur 75.2→68.6). `EQUAL_WEIGHT_REGIONS` restored hsr/bellandur influence → 84.8 % leader + lower OVER (977).
 8. **SAGPool collapse (Phase 1 iter18):** hierarchical pooling permanently drops nodes → spatial-path discontinuity → floods trapped on retained coarse nodes. **Hard constraint: never destructively pool; production HydroGINE preserves full topology** (paper ablation: SAGPool coarse = collapse vs Standard 6.82 cm vs Full Topology 2.39 cm).
 9. **log1p target + aggressive tail weights interact:** the `expm1` decode amplifies log-domain error on deep basins, which is *why* Huber-in-log-space + deep-target weighting must be balanced, not maximized.
 10. **Chunk-size / hardware coupling:** `N_CHUNK` 25000 → 20000 after an RTX 4060 TDR/cublas backward fault; reproducibility pinned with `SEED=42` throughout.
-

Appendix — Files referenced

- `production_v4.py` (527 ln) — final predictor + `EnsembleFloodPredictorV4`
- `train_hydro_gine_v5_0.py` (556 ln) — v5.11 final training engine
- `master_pinn_engine.py` (159 ln), `active_pinn_pipeline.py` (144 ln) — PINN prototypes
- `build_physics_dataset.py`, `build_expanded_physics_dataset.py`, `create_multi_pyg_dataset.py`, `create_testcity_dataset.py`, `create_combined_dataset.py` — dataset/feature builders
- `benchmark_all_versions.json`, `benchmark_final_v58.json`, `benchmark_ensemble_v56_v58.json`, `benchmark_live_match_results.json`, `competition_benchmark_verified.json`, `round2_live_benchmark_50.json`, `round2_live_benchmark_100.json` — metrics
- `Model_Evolution_Trace.md` — the subagent archaeologized lineage (this doc is its peer-reviewable form)

- `data/real_bengaluru_oct2024_incidents.json`, `real_world_validation_result.json` — the real-world validation set and its live result (see §11); `historical_validation_result.json` is stale/malformed
- `data/canonical_metrics.json` — canonical, reproducible metric set (50/100 mm/hr, capture, latency)
- `doc/UrbanFLOW_Research_Paper.md`, `doc/System_Architecture_and_Model_Specification.md` — peer-facing docs describing v5.0/v5.8 era; **reconcile with this final-version document.**